

Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Специальность «Программная инженерия»

Кафедра программного обеспечения информационных технологий

Учебная дисциплина «Метрология, стандартизация и сертификация в
информационных технологиях»

Отчет

по практической работе №2а
«Метрики Джилба»

Подготовили:

Горьковой Н. гр. 410902
Синютин Н. гр. 410902

Проверил:

Болтак С.В.

Минск 2026

Цель: Изучить метрики сложности потока управления

Задание: Написать программу-парсер. Парсер может быть реализован на любом языке программирования (TypeScript) и должен иметь графический интерфейс. Для анализа взять подготовленную программу на заданном языке (Python). Результат работы парсера: вывод на экран абсолютной сложности, относительной сложности и максимального уровня вложенности.

Язык: Python

Анализируемый код на языке Python:

```
def process_matrix(matrix, limit):
    rows = len(matrix)
    cols = len(matrix[0]) if rows > 0 else 0
    total = 0
    evens = 0
    odds = 0
    zeros = 0

    for i in range(rows):
        for j in range(cols):
            value = matrix[i][j]
            if value == 0:
                zeros += 1
            elif value % 2 == 0:
                evens += 1
            else:
                odds += 1
            total += value

    diagonal = 0
    k = 0
    while k < rows and k < cols:
        item = matrix[k][k]
        if item != 0:
            match item % 4:
                case 0:
                    diagonal += item
                case 1:
                    diagonal -= item
                    if item > limit:
                        diagonal += limit
                case 2:
                    step = 0
                    while step < 2:
                        diagonal += step
```

```

        step += 1
    case _:
        diagonal += 1
    k += 1

for row in matrix:
    row_sum = 0
    for value in row:
        if value > 0:
            row_sum += value
    if row_sum > limit:
        total -= row_sum
    else:
        total += row_sum

if evens > odds:
    result = total + diagonal
elif evens == odds:
    result = total
else:
    result = total - diagonal

match zeros:
    case 0:
        result *= 2
    case 1:
        result += limit
    case _:
        result -= zeros

return result

```

Ручной подсчет значений

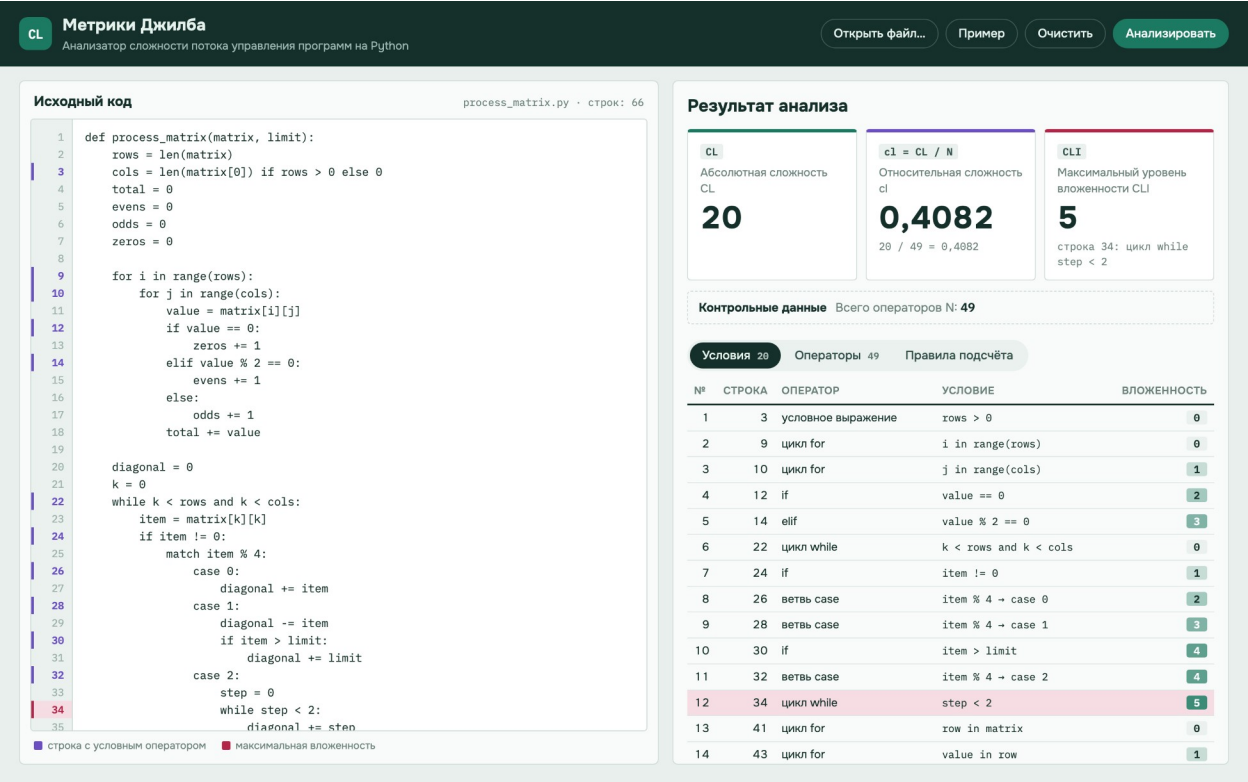
Абсолютная сложность $CL = 20$

Общее число операторов $N = 49$

Относительная сложность $cl = 20/49 = 0,4082$

Максимальный уровень вложенности $CLI = 5$

Результат подсчета программы представлен на рисунке 1



Результат анализа

CL

Абсолютная сложность CL

20

cl = CL / N

Относительная сложность cl

0,4082

20 / 49 = 0,4082

CLI

Максимальный уровень вложенности CLI

5

строка 34: цикл while step < 2

Контрольные данные

Всего операторов N: 49

Условия 20

Операторы 49

Правила подсчёта

№	СТРОКА	ОПЕРАТОР	УСЛОВИЕ	ВЛОЖЕННОСТЬ
1	3	условное выражение	rows > 0	0
2	9	цикл for	i in range(rows)	0
3	10	цикл for	j in range(cols)	1
4	12	if	value == 0	2
5	14	elif	value % 2 == 0	3
6	22	цикл while	k < rows and k < cols	0
7	24	if	item != 0	1
8	26	ветвь case	item % 4 == case 0	2
9	28	ветвь case	item % 4 == case 1	3
10	30	if	item > limit	4
11	32	ветвь case	item % 4 == case 2	4
12	34	цикл while	step < 2	5
13	41	цикл for	row in matrix	0
14	43	цикл for	value in row	1

Рисунок 1 – результат подсчета программы

Схема алгоритма представлена на рисунках 2 и 3.

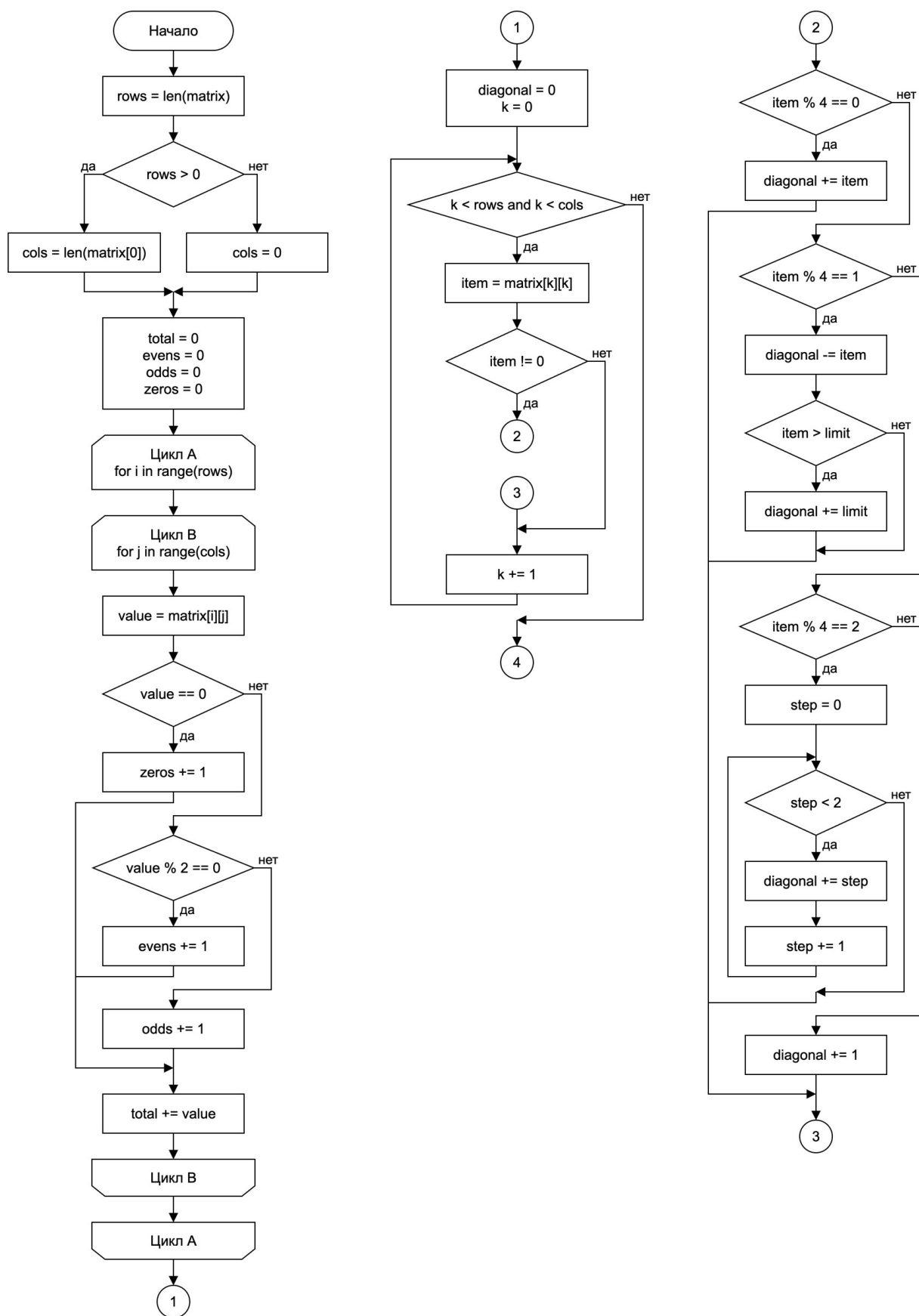


Рисунок 2 – схема алгоритма (лист 1)

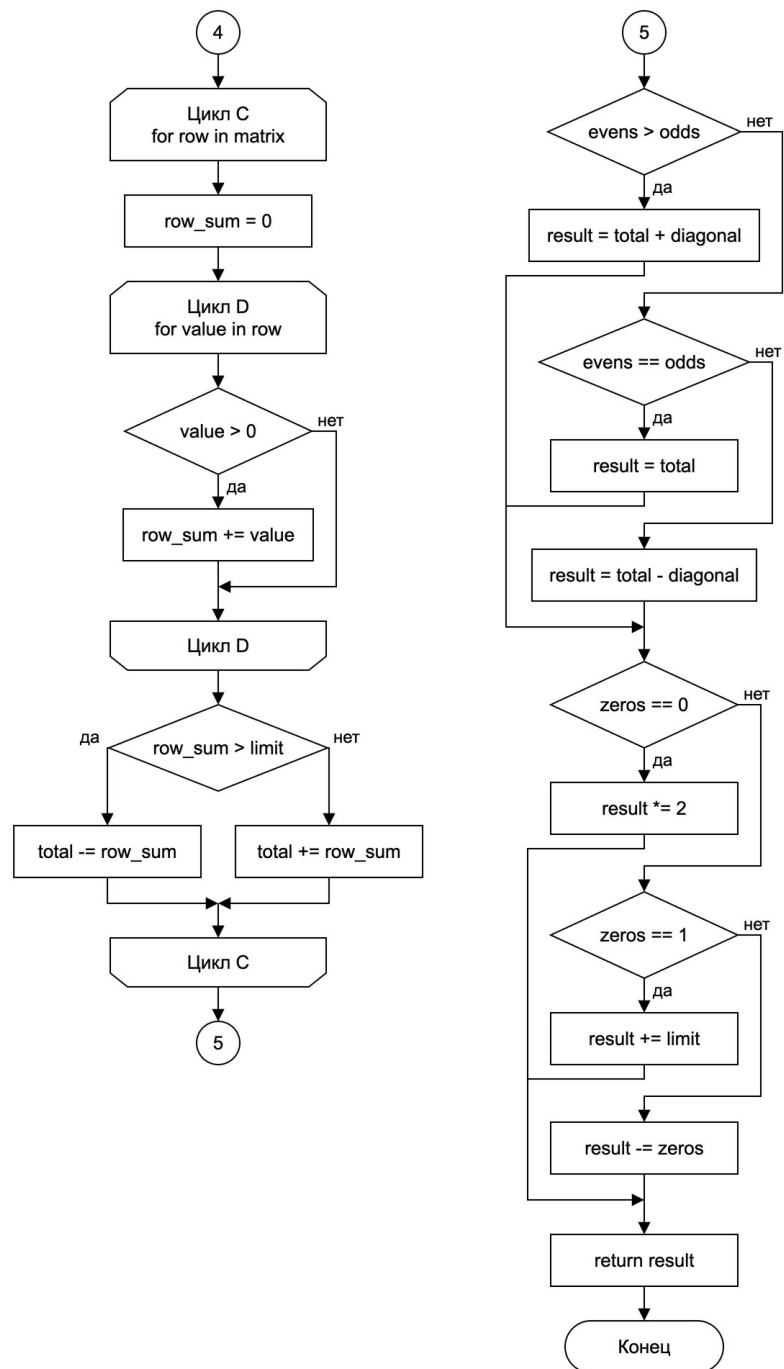


Рисунок 3 – схема алгоритма (лист 2)